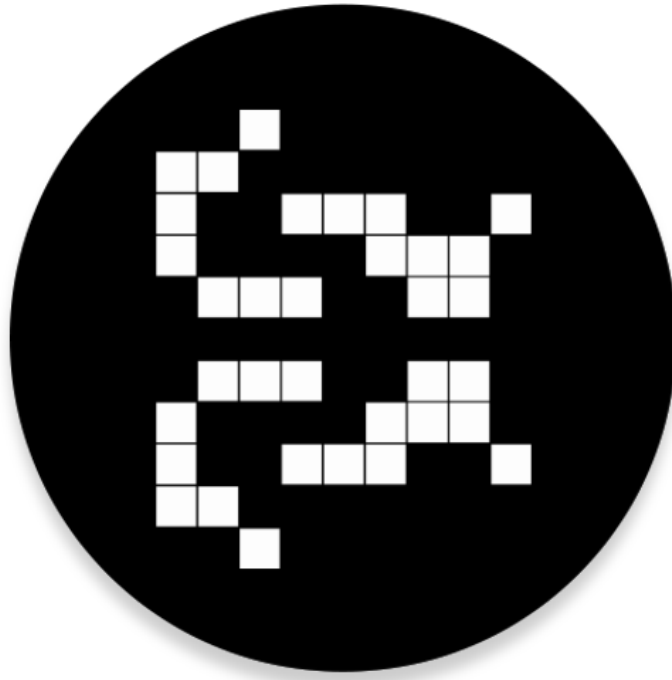




PROJET - JAVA

Automates Cellulaires

Élèves :

El Varougou HAIBALLA

Aïssa PANSAN

Enseignant :

Alexandre

DUPONT-BOUILLARD

John CHAUSSARD

16 avril 2023

Send you our best regards,

Sommaire

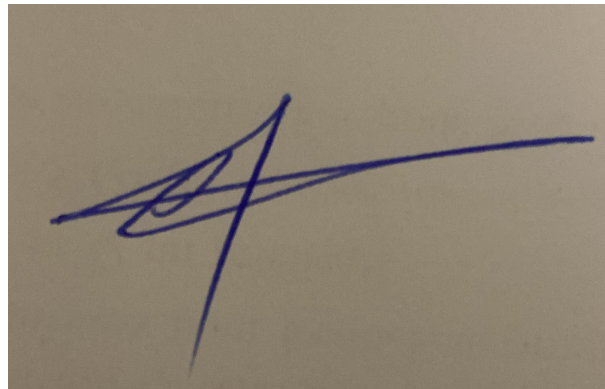
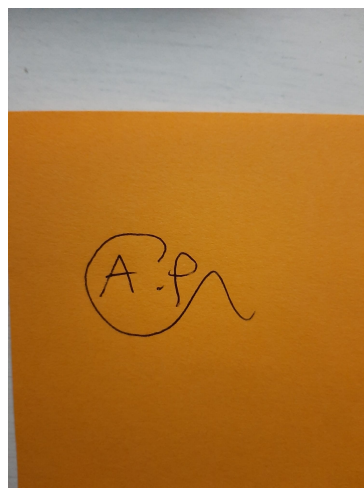
1	Engagement de non plagiat	3
2	Introduction générale	4
2.1	Explication rapide du programme	4
3	Architecture générale du programme	6
4	Réalisation du programme principale	7
4.1	Implémentation de la grille	7
4.1.1	Principe	7
4.1.2	Explication de la génération de grille récursive en détail	8
4.1.3	Difficulté rencontrée	11
4.1.4	getCellule	12
4.1.5	Choix des ArrayList	12
4.2	Parcours de la grille	12
4.2.1	Logique d'itération	12
4.2.2	Utilisation d'itérateurs	13
4.3	Génération des coordonnées	14
4.3.1	Difficulté rencontrée	16
4.4	Voisinage d'une cellule	19
4.4.1	Principe	19
4.5	Implémentation de la solution	20
4.5.1	Exemple	23
5	Implémentation des règles	23
5.1	Structure des opérateurs	24
5.2	Cas particulier des opérateurs ternaire et unaire	25
5.3	Principe de lecture d'une règle	26
5.4	Implémentation de la solution : Classe TraitRegle	27

5.5	Autres	29
6	Fichier de configuration	31
6.1	Rapide introduction à l'XML et avantages d'un tel type de fichiers	31
6.2	Exploitation au niveau du Main.java	32
7	Interface graphique	34
8	Exécution du Main.java	37
9	Résultats	39
9.1	Cas 1D : Triangle de Sierpiński	39
9.2	Cas 2D	40
10	Conclusion générale	41
11	Annexe	43
11.1	Diagramme d'activité de TraitRegle	43

1 Engagement de non plagiat

Nous, soussigné(e)s Aïssa Pansan et Farouk Haiballa, étudiants en 1er année d'école d'ingénieur à Sup Galilée, déclarons être pleinement conscient(e)s que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée, tout comme l'utilisation d'outils d'Intelligence Artificielle pour générer une partie de ce rapport ou du code associé. En conséquence, nous déclarons que ce travail ne comporte aucun plagiat, et assurons avoir cité explicitement, à chaque fois que nous en avons fait usage, toutes les sources utilisées pour le rédiger. Fait à Villetaneuse , le 15/06/2024

Signatures :

A handwritten signature in blue ink on a light brown background. The signature is stylized, featuring a large, sweeping 'A' shape with a long horizontal stroke extending to the right.A handwritten signature in black ink on an orange background. The signature is enclosed in a circle and appears to be 'A.P.' followed by a flourish.

2 Introduction générale

Nous avons l'honneur de vous présenter le rapport de notre projet d'Automates Cellulaire, réalisé en tant qu'étudiants en première année d'ingénierie en informatique. Ce projet s'inscrit dans le cadre de notre formation en Programmation Orientée Objet, qui comprend l'utilisation du langage Java. Ce projet a été l'occasion pour nous de mettre en pratique les concepts appris en classe.

Nous sommes fiers de partager avec vous les résultats de notre travail et espérons qu'il démontrera notre compréhension des concepts que l'on a étudiés durant ce semestre.

Ce projet vise à implémenter des Automates Cellulaires. Ce projet comporte de nombreux enjeux, comme l'implémentation d'une grille N-dimensionnelle, le calcul de cellule voisine, l'implémentation de règle pour la vie et la mort des cellules, l'implémentation d'une interface graphique et l'exploitation d'un fichier de configuration en XML. Pour répondre à ces différentes problématiques, il nous a été nécessaire de choisir le bon environnement de travail, afin de pouvoir coder, mais aussi déboguer notre programme de manière efficace. Notre choix s'est ainsi porté sur l'IDE de JetBrains **IntelliJ Idea**. Enfin, ce rapport présente les détails de notre travail, expliquant les décisions prises et les difficultés rencontrées durant son élaboration. Il a pour objectif de synthétiser notre démarche de résolution en détaillant succinctement les étapes clés du projet.

2.1 Explication rapide du programme

Un automate cellulaire est une grille composée d'un ensemble de cellules. Ces cellules, en fonction de règles préétablies, peuvent soit mourir (et donc se colorier en blanc dans une interface graphique), continuer à vivre, ou naître (ce qui est traduit par une couleur noire). Ce programme est directement inspiré du jeu de la vie inventé par **John Conway** en 1970. En partant d'une configuration de base, on peut arriver à des représentations spectaculaires en 1D ou 2D.

Dans le cadre de notre projet, nous devons implémenter une grille de dimension inconnue à l'avance, cette dernière doit être choisie par l'utilisateur, pour générer une telle

grille, il a été nécessaire d'utiliser de la **récurtivité**. Si la grille dépasse la dimension 2, l'affichage doit être sur une coupe de la grille spécifiée par l'utilisateur. Ensuite, le programme doit pouvoir calculer pour n'importe quelle cellule toutes les cellules voisines en fonction du type de voisinage choisie, afin d'y pouvoir évaluer ou appliquer les règles du jeu de la vie. Pour ce faire, nous avons utilisé **des concepts d'abstraction** afin de générer n'importe quel voisinage. Par la suite, le programme doit pouvoir avoir un système de règles pour pouvoir faire évoluer l'automate au cours du temps. Cela a été possible, en utilisant le concept du **polymorphisme** et grâce à l'implémentation d'un **parser**. Enfin, il a été nécessaire d'apprendre à faire interfacer un fichier XML et un programme Java, afin de permettre une description limpide pour l'utilisateur de la configuration de l'automate cellulaire.

En plus de ces classes-là, il existe d'autre classe telle que la classe permettant de générer l'interface graphique, les classe de génération d'exception et enfin la programme principale **Main.java**.

4 Réalisation du programme principale

4.1 Implémentation de la grille

Notre programme doit dans un premier temps pouvoir générer un tableau de grille à dimension fixé par l'utilisateur. Cette dernière pouvant allégrement dépasser la dimension 2, il est impossible d'utiliser les structure de tableaux classique. C'est pourquoi nous avons très vite utilisé le principe de la récursivité pour pouvoir construire une grille indépendamment de la dimension de base.

4.1.1 Principe

Pour gérer une telle structure, il a fallu utiliser le principe d'encapsulation de Java. Deux classes qui héritent de la même classe appartiennent à une même famille. Dans notre cas, nous avons une classe abstraite **Grille**, une classe **TableauDynamiqueND** et une classe **Cellule**. Ces deux dernières classes héritent de **Grille**.

La classe **Grille** va ainsi contenir :

- Un tableau d'Integer **dimension** qui permet de connaître la dimension de la grille à générer (Ce champ est plus pratique pour **TableauDynamiqueND**
- Un entier **taille** qui précise la taille de la dimension courante, il est équivalent à `dimension.get(0)`
- Un état **status** qui définit si une cellule est en vie ou non.

La classe **TableauDynamiqueND** contient :

- Une ArrayList de grille qui contient, c'est ce tableau qui permet de générer la grille.
- Un entier **level** qui précise dans quel niveau de l'arborescence à l'initialisation.

L'idée est ainsi d'avoir une classe tampon qui ne servira uniquement à contenir d'autre

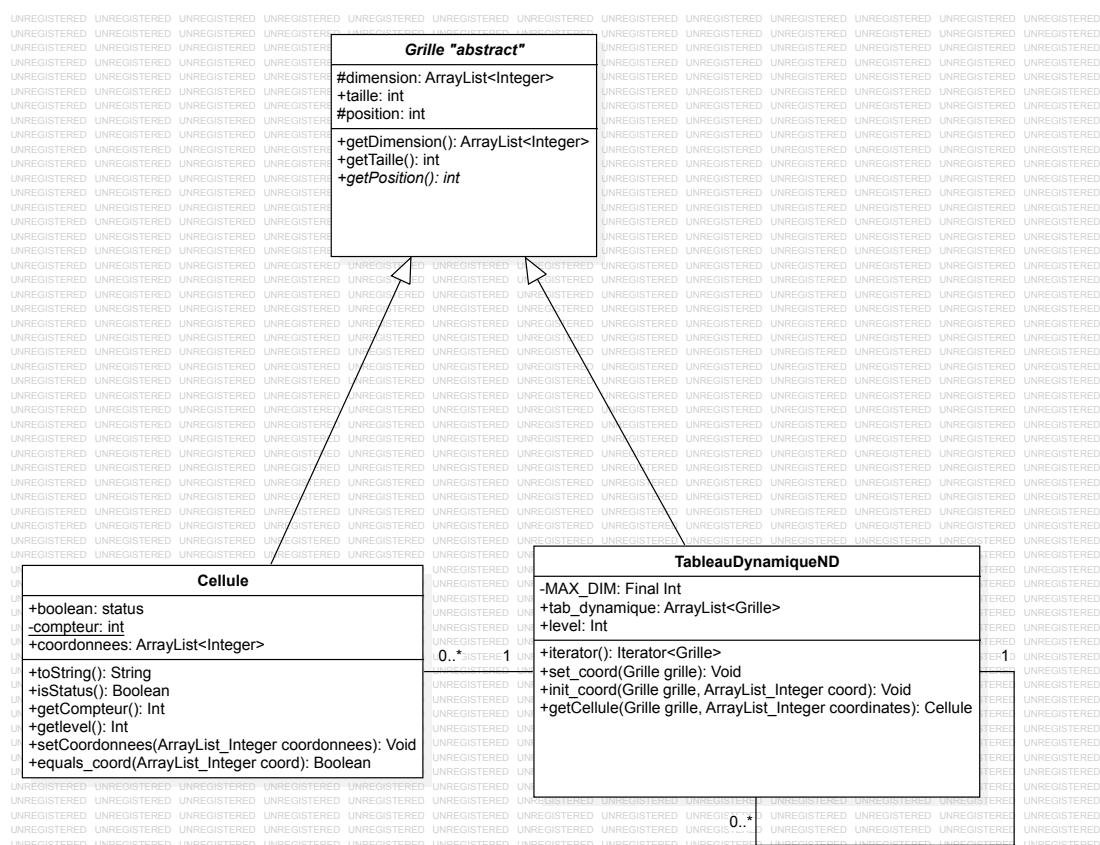


FIGURE 2 – Diagramme de classe de la grille

objet du même type **TableauDynamiqueND** lors d'un appel récursif au constructeur. Le cas de base de la récursivité est d'allouer dans notre classe **TableauDynamiqueND** un tableau de **Grille** qui contient en réalité des **Cellules**. Ainsi, on peut générer dynamiquement la grille sans avoir besoin de connaître au préalable la dimension à l'avance.

4.1.2 Explication de la génération de grille récursive en détail

Pour générer la grille, on va utiliser des `ArrayList` qui représente la dimension de la grille. Ainsi :

```

ArrayList<Integer> dim = new ArrayList<>();

dim.add(4);

dim.add(3);

dim.add(2);

```

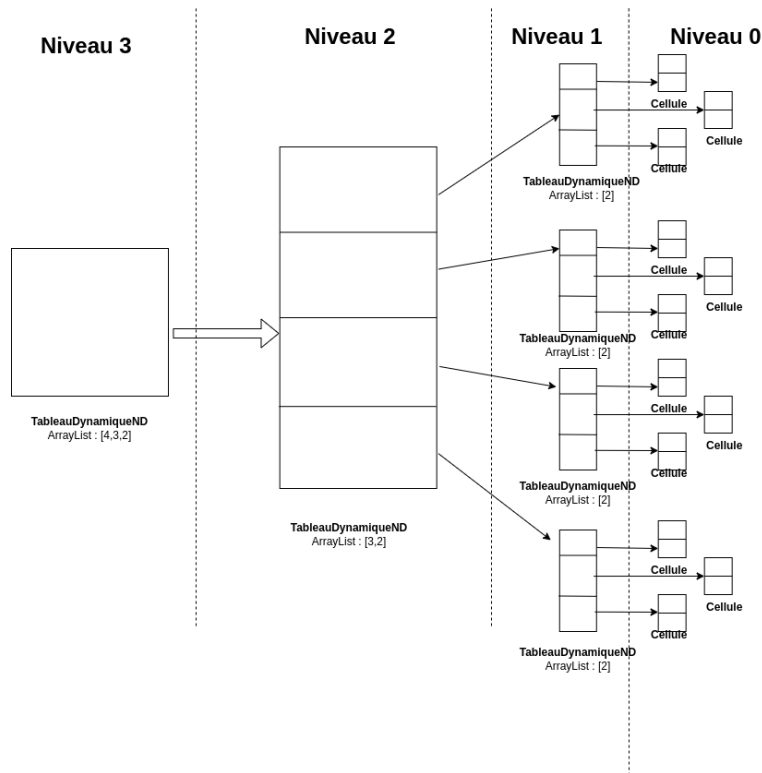


FIGURE 3 – Schéma de la structure de la grille, pour une grille 3D

Est une grille de dimension 3, avec 4 cases en abscisse, 3 cases en ordonnées et 2 cases en hauteur.

On va passer cette ArrayList dans un premier constructeur par défaut qui prend en extrait la dimension, via la méthode **size** de ArrayList ;

Ensuite, on appelle un second constructeur qui va réellement générer la récursivité, ce dernier est private, afin d'éviter que l'utilisateur ne provoque des erreurs en passant des mauvais arguments.

```
private TableauDynamiqueND ( int d, ArrayList<Integer> taille_tab) throws
```

```
Exception1 {
```

```
    On initialise notre tableau de grille. La taille_tab du tab est celle de
```

```
    la 1er taille_tab du tab taille_tab
```

```
    tab_dynamique = new ArrayList<>(taille_tab.getFirst());
```

```
    if( taille_tab.size() > MAX_DIM)
```

```
    {
```

```
        System.out.println("Tableau DynamiqueND excedes maximum taille_tab");
        throw new IndexOutOfBoundsException();
    }

    //Partie réursive
    if( d == 1 )
    {
        taille = taille_tab.getFirst();
        for(int i = 0; i < taille; i++)
        {
            Cellule c = new Cellule();
            tab_dynamique.add( c );
            System.out.println("Nombre de cellule :" + c.getCompteur());
        }
    }
    else
    {
        ArrayList<Integer> copy = new ArrayList<>(taille_tab);
        taille = taille_tab.getFirst();
        if( taille == 1)
        {
            throw new Exception1();
        }
        copy.removeFirst();
        for(int i = 0; i < taille; i++)
        {
            try
            {
                TableauDynamiqueND new_tab = new TableauDynamiqueND(d-1,copy);
                tab_dynamique.add( new_tab );
            }
        }
    }
}
```

```
        catch (Exception e)
        {
            System.out.println("Erreur case : " + i + " - Dimension : " +
                               d + e );
        }
    }
}
```

À chaque étape de la récursivité, on va générer un tableau de Grille en fonction de la dimension du 1er indice du tableau d'Integer. Ensuite, on va récupérer le premier élément du tableau d'Integer dans une variable **taille**, cette dernière va nous servir à nous repérer dans la hiérarchie et donc à continuer à faire des appels récursifs. On va aussi supprimer le 1er élément du tableau d'Integer pour connaître les prochaines dimensions à générer. C'est ainsi qu'on va rappeler le constructeur pour aller plus loin dans l'arborescence, tout en veillant à décrémenter la dimension de 1. Lors du retour de l'appel récursif, on ajoute à notre **tab_dynamique** l'objet **new_tab** Le tableau nommé **tab_dynamique** peut contenir soit des classes **TableauDynamiqueND**, soit des **Cellules**. Le cas d'arrêt de la récursivité est quand on atteint une dimension de 1. On va cette fois-ci pouvoir générer des **Cellules** qui seront ajoutées au tableau de grille de notre objet **TableauDynamiqueND** de dimension 1.

4.1.3 Difficulté rencontrée

Lors de notre 1er implémentation, nous n'avions pas pris conscience du fait que la modification d'un tableau d'Integer à un appel de la récursion aurait aussi un impact sur toute la hiérarchie. En conséquence, notre 1er version, ne fonctionnait qu'avec des tableaux 2D, au-delà le programme essayait d'accéder à des cases non définies. Pour remédier à ce problème, nous avons décidé de cloner l'ArrayList avant chaque appel récursif. De cette

manière, chaque appel modifie sa propre instance, exactement de la même manière qu'avec des tableaux classiques.

4.1.4 getCellule

4.1.5 Choix des ArrayList

Dès le départ de mon implémentation, j'ai commencé à utiliser des ArrayList. Ce choix s'explique par le fait que les ArrayList sont très bien adaptés en cas de modification de leur taille. En effet, l'usage d'un tableau classique aurait nécessité la recopie de chaque int, sauf du 1er. Pour pouvoir, éliminer le 1er terme. En termes de complexité de code, l'ArrayList permet d'être plus concis, cela dit, il n'est pas dit qu'il en soit de même pour la complexité Algorithmique.

4.2 Parcours de la grille

Après avoir généré la grille, il nous faut la parcourir afin de réaliser plusieurs opérations sur cette dernière (Initialisation des coordonnées, recherche d'une cellule, application de règle...)

4.2.1 Logique d'itération

Notre parcours s'inspire de la logique du DFS (Deep First Search) que j'ai aussi pu coder dans le projet **Puissance4**. Partant de l'objet principal **TableauDynamiqueND**, on veut pouvoir itérer sur chacune des case de **tab_dynamique**. On veut aussi pouvoir accéder à des sous-tableaux tant qu'on n'a pas atteint un objet **TableauDynamiqueND** de niveau 1 (Le dernier niveau qui contient des cellules dans **tab_dynamique**). Au niveau du tableau de dimension n, on va itérer sur la case suivant une fois qu'on a itéré sur toutes les cases d'un tableau de dimension n-1. Pour gagner en lisibilité et en abstraction, j'ai aussi décidé d'utiliser des itérateurs.

4.2.2 Utilisation d'itérateurs

Dans Java, les itérateurs sont un outil puissant pour pouvoir itérer sur une structure de données sans même en connaître ses fondements. Ce qui est très pratique pour tout développeur qui ne connaît pas l'implémentation initiale de la structure de données. Une classe peut utiliser des itérateurs lorsqu'elle implémente l'interface `Iterable`. On peut ainsi définir une méthode **iterator()** qui va instancier un objet de type `Iterable`, dans notre cas, cela sera `Iterable<Grille>`. Ensuite, à chaque fois que l'on voudra utiliser un itérateur, on utilisera cette méthode

```
public Iterator<GrilleV2> iterator()
{
    return new parcoursTab(this.taille, this.tab_dynamique);
}

...

Iterator<GrilleV2> iter = ((TableauDynamiqueND)grille).iterator();
```

Cela dit, le travail, ne s'arrête pas là. Si l'on souhaite itérer sur la grille, il faut pouvoir le spécifier. La classe **parcoursTab** permet d'explicitement la manière d'itérer sur la grille. Cette classe va implémenter les interfaces **Iterable<Grille>** et **Iterator<Grille>**. L'implémentation de cette dernière interface exige de redéfinir deux méthodes :

- **hasNext()** une méthode qui renvoie un booléen pour vérifier s'il existe d'autres éléments à parcourir
- **next()** qui renvoie un objet de type **Grille**

Lors de son instantiation, l'itérateur **parcoursTab**, a en attribut la taille du tableau sur lequel on itère et un tableau de grille, qui correspond à celui de **tab_dynamique**. Il possède aussi un champ **index** qui est initialisé à 0, cet index fait office de curseur de lecture sur un tableau. La méthode **hasNext()** va simplement vérifier si l'index ne dépasse pas la taille du tableau. La méthode **next** va renvoyer un élément du tableau de grille en fonction de l'index, qui est incrémenté à chaque appel.

hasNext :

Entrées : void

Sortie : boolean

Fonctionnement : La fonction va simplement vérifier si l'index ne dépasse pas la taille du tableau.

4.3 Génération des coordonnées

La génération des coordonnées utilise directement les capacités de la classe **parcours-Tab**. Le parcours se fait en plusieurs étapes :

1. On commence par générer un objet **Iterator<Grille>**
2. On teste si l'on est au niveau 1 de la hiérarchie
3. Si oui, on ajoute un 0 dans le tableau de coordonnées, on va ensuite parcourir toutes les cellules et leur ajouter la coordonnée qui a été passé d'appel en appel récursif, tout en veillant à incrémenter la dernière coordonnée.
4. Sinon, on ajoute un 0 dans le tableau de coordonnées
 - (a) Ensuite, pour chaque tour de boucle, on teste via **hasNext** s'il reste des case à parcourir dans **tab_dynamique**
 - (b) Si oui, on va récupérer le sous tableau via **next()**, dans le même temps, on copie le tableau **coord**.
 - (c) On écrit la coordonnée qui correspond à l'opération **coord.size() -1**, dans le tableau **coord**
 - (d) On rappelle récursivement, en envoyant **sub_tab** et la copie de **coord**, on incrémente i après chaque tour de boucle afin de décaler l'initialisation du tableau de coordonnées

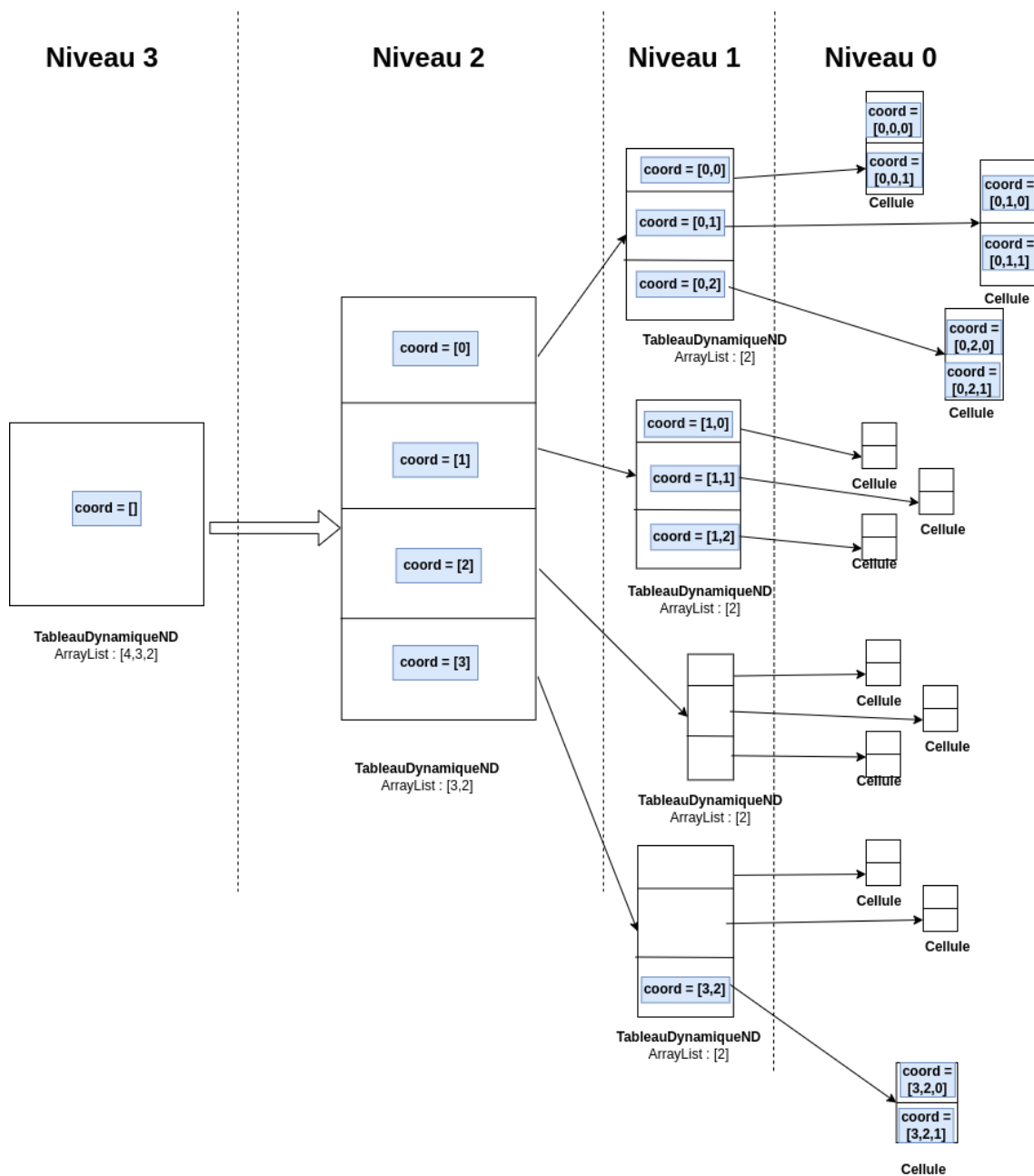


FIGURE 4 – Illustration du remplissage des coordonnées pour une grille [4,3,2]

4.3.1 Difficulté rencontrée

L'implémentation de cette méthode fut très fastidieuse, à cause de notre méconnaissance des itérateurs, mais plus encore de la récursivité en Java. Lors de notre implémentation, nous constatons que notre itérateur bouclait à l'infini. En définitive, il faisait du surplace, malgré l'allocation d'un nouvel itérateur à chaque étape de la récursivité. Après une analyse approfondie de l'exécution et grâce au débogueur **d'IntelliJ Idea**, on s'est rendu compte de l'importance de déclaration des termes. Au début, tous les champs de la méthode `init_coord` étaient relatifs à la classe `TableauDynamiqueND` et donc à cet objet généré dans le `Main.java`. Le problème étant qu'à chaque itération, l'accès à un champ référait à l'objet appelant (et non à l'objet issu de la récursivité). Cette distinction, bien que banale, fut très important pour garantir le fonctionnement de la méthode. On peut illustrer les conséquences d'une telle confusion dans une méthode `DFS_exemple` pour grille de type `[4,3]`.

```
public void DFS_search_example(GrilleV2 grille)
{
    //Champ de l'objet recursif
    System.out.println("Grille dans un tab de dimension " +
        grille.getDimension()+
        " Niveau : " + grille.getLevel()
        + " Mon tab_dimension a : " +
        ((TableauDynamiqueND)grille).tab_dynamique.size() + " elements");

    //Champ de l'objet d'instance (L'objet qui appelle DFS_search dans le
    main)
    System.out.println("Grille dans un tab de dimension " +
        this.getDimension()
        + " Niveau : " + this.getLevel()
        + " Mon tab_dimension a : "
        + tab_dynamique.size()
        + " elements");
```

```
if(grille.getLevel() == 1)
{
    System.out.println("Bout de course\n");
}
else
{
    for (int i = 0; i < grille.getDimension().get(0); i++)
    {
        System.out.println("Appel recursive");
        Grillev2 sub_tab =
            ((TableauDynamiqueND)grille).tab_dynamique.get(i);
        DFS_search_example(sub_tab);
    }
}
}

/// Output
```

Grille dans un tab de dimension [4, 3] Niveau : 2 Mon tab_dimension a : 4

elements

Grille dans un tab de dimension [4, 3] Niveau :2 Mon tab_dimension a : 4

elements

Appel recursive

Grille dans un tab de dimension [3] Niveau : 1 Mon tab_dimension a : 3 elements

Grille dans un tab de dimension [4, 3] Niveau :2 Mon tab_dimension a : 4

elements

Bout de course

Appel recursive

Grille dans un tab de dimension [3] Niveau : 1 Mon tab_dimension a : 3 elements

Grille dans un tab de dimension [4, 3] Niveau :2 Mon tab_dimension a : 4
elements

Bout de course

Appel recursive

Grille dans un tab de dimension [3] Niveau : 1 Mon tab_dimension a : 3 elements

Grille dans un tab de dimension [4, 3] Niveau :2 Mon tab_dimension a : 4
elements

Bout de course

Appel recursive

Grille dans un tab de dimension [3] Niveau : 1 Mon tab_dimension a : 3 elements

Grille dans un tab de dimension [4, 3] Niveau :2 Mon tab_dimension a : 4
elements

Bout de course

Le 1er **System.out.println** évolue bien en fonction de l'objet récursif quand on l'appelle avec la grille, mais pas le deuxième quand on utilise directement les champs de la classe.

4.4 Voisinage d'une cellule

Pour que le programme fonctionne correctement, on a besoin de savoir quel est le voisinage d'une cellule. Le voisinage d'une cellule dépend de plusieurs paramètres, notamment la dimension de la grille choisie et type de voisinage. Bien que l'utilisateur puisse lui-même renseigner son propre voisinage, il est d'abord important de coder un certain nombre de voisinages dits classique, voisinages G0,G2,G4,G8,G6 et G26. Pour ce faire, nous allons utiliser une classe abstraite **Neighbor**.

4.4.1 Principe

Les différents types de voisinages fonctionnent de la même manière. L'unique différence est l'implémentation de voisin admissible à une case donné. Afin d'éviter de répéter le même code dans chaque classe, on utilise une classe abstraite **Neighbor**.

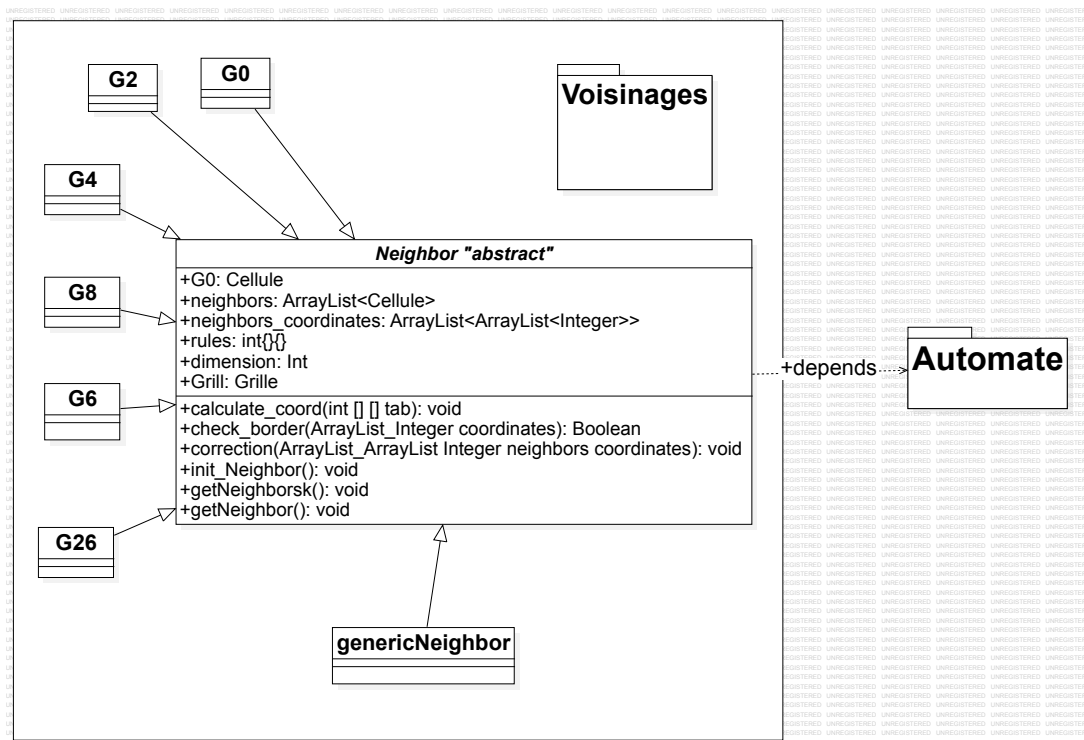


FIGURE 5 – Diagramme de classe du Voisinage

Les voisinages spécifiques (G2,G4,...) vont renvoyer un tableau 2D d'Int qui contiendra les index des cellules voisines. Par exemple, pour **G4**, les cellules voisines sont situées à gauche, à droite, en haut en bas de **G0**. Cela est traduit par le tableau : $[\{1, 0\}, \{-1, 0\}, \{0, 1\}, \{0, -1\}]$.

Dans le constructeur de **G4**, on va faire un appel au constructeur de **Neighbor** via **super** en envoyant ce tableau d'Int qui sera le champ **Rule** de la classe **Neighbor**. La classe **Neighbor** va alors effectuer les opérations suivantes pour générer le voisinage d'une cellule :

1. Initialisation des champs via le constructeur
2. Calcul de toutes les coordonnées possible de **G0** en faisant une somme vectorielle entre les coordonnées de **G0** et le tableau **Rules**
3. Éliminations des coordonnées hors grille
4. Remplissage du tableau **Neighbor** par itération sur les coordonnées trouvés

Ces opérations seront effectuées sur les attributs de la classe **Neighbor**

- **G0** : Cellule pour laquelle, on va étudier le voisinage
- **neighbors** : ArrayList de **Cellules**, elle contient l'ensemble des cellules voisines de **G0**
- **neighbors_coordinates** : l'ArrayList à deux dimensions, elle contient les coordonnées des voisins potentiels de **G0**
- **Rule** : Tableau 2D, qui contient les index des voisinages
- **Dimension** : Spécifié la dimension du voisinage (1D,2D,3D,...)
- **Grille** : La classe **Neighbor** a besoin d'une référence de la grille pour pouvoir accéder aux cellules voisines

Dans le constructeur de la classe **Neighbor**, on va récupérer la cellule **G0**. On va ensuite initialiser l'ArrayList **neighbor** et **neighbors_coordinates**. De plus, on va associer à notre objet **Neighbor** le tableau 2D **Rule** spécifié par les **Gk** et on va lui associer la grille. Enfin, on lance la méthode **makeNeighbor()** qui permet de directement initialiser le tableau de voisin. Cette méthode en appelle 3 autres.

4.5 Implémentation de la solution

calculate_coord :

Entrées : `int[][] tab`

Sortie : void

Fonctionnement : La méthode va calculer les coordonnées potentiels de **G0** à l'aide du tableau **tab**. On boucle deux fois à l'aide d'un `forEach` dans chacune des case de **tab**. On y instancie une `ArrayList` d'`Integer`, appelé **coord** qui copie les coordonnées de **G0** et qu'on ajoute à **neighbors_coordinates**. Ensuite, on va itérer sur les composantes de **tab** pour pouvoir modifier la coordonnée de **coord** en effectuant la somme des coordonnées de **G0** avec celle de **tab**. Pendant, toutes ces itérations, on veille à incrémenter un **index_i** pour pouvoir accéder à la bonne coordonné de **G0**

```
protected void calculate_coord(int[] [] tab)
{
    for (int[] T : tab)
    {
        ArrayList<Integer> coord = new ArrayList<>(G0.getCoordonnees());
        neighbors_coordinates.add(coord);
        int index_i = 0;
        for (int index : T)
        {
            if (index_i < coord.size())
            {
                int new_coord = coord.get(index_i) + index;
                coord.set(index_i, new_coord);
            }
            else
            {
                throw new IndexOutOfBoundsException("Index " + index_i + "
                    out of bounds for length " + coord.size());
            }
            index_i++;
        }
    }
}
```

}

On peut résumer la logique de cette méthode via ce diagramme d'activité

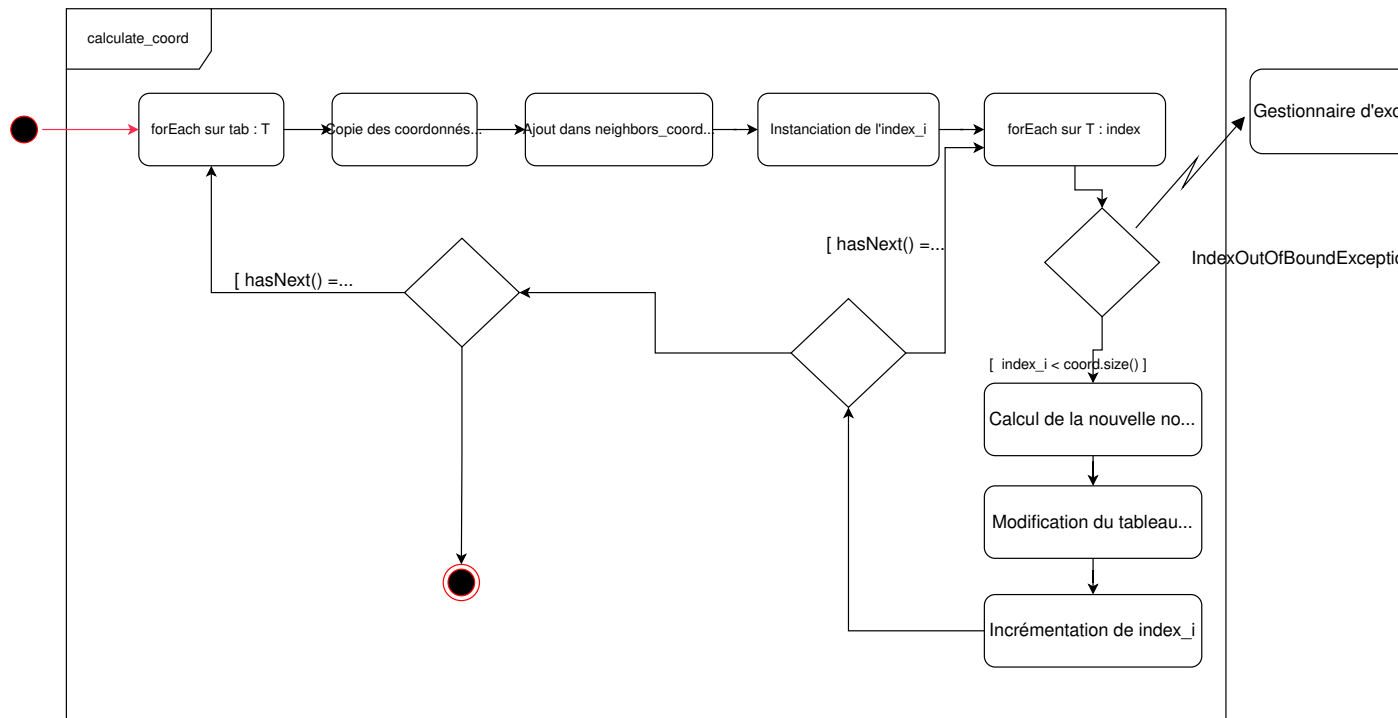


FIGURE 6 – Diagramme de classe du Voisinage

Après avoir obtenu tous les voisins possibles, il faut éliminer ceux qui sont de la grille. C'est le rôle des méthodes **correction** et **check_border**

check_border :

Entrées : `ArrayList<Integer> coordinates`

Sortie : `Boolean`

Fonctionnement : La méthode va itérer sur chacune des composantes de **coordinates**. On va vérifier, si la coordonner est bien compris entre 0 et la dimension de la grille via **Grille.dimension.get(border)**. La limite supérieure est en réalité le tableau d'Integer que contient toute grille et qui permet de connaître la dimension de grille générale. On se décale à chaque fois de **border** pour lire la bonne dimension de l'ArrayList **dimension**. Si la cellule est dans les bornes, on renvoie **true**, sinon **false**.

correction :

Entrées : `ArrayList<ArrayList<Integer>> neighbors_coordinates`

Sortie : `void`

Fonctionnement : La méthode va utiliser un itérateur sur le tableau `neighbors_coordinates`. On va appliquer la méthode `check_border` sur chacun des éléments de `neighbors_coordinates`. À chaque fois qu'une coordonnée est incorrect, on la supprime via `iterator.remove()`. On obtient à la fin un tableau `neighbors_coordinates` contenant des coordonnées valide. On a utilisé des itérateurs, pour éviter de générer l'exception `ConcurrentModificationException` qui se manifeste quand on itère sur élément d'un tableau et qu'on cherche en même temps à le supprimer.

On peut enfin récupérer toutes les cellules issues de `neighbors_coordinates`

init_Neighbor() :

Entrées : `void`

Sortie : `void`

Fonctionnement : La méthode va parcourir toute l'ArrayList `neighbors_coordinates`, récupérer la cellule via `getCellule` et l'ajouter au tableau `neighbors`

Par ailleurs, l'utilisateur peut plus tard spécifier son propre voisinage en utilisant la classe `genericNeighbors`. Pour ce faire, il suffit de lui passer le voisinage spécifique en argument du constructeur.

4.5.1 Exemple

5 Implémentation des règles

L'implémentation des règles est une partie essentielle du projet, car elle va définir la cohérence de l'automate cellulaire. Dans un premier temps, il nous a fallu réfléchir à une hiérarchie des classes qui permettent d'utiliser tous les connecteurs introduit dans notre sujet. Ensuite, il a fallu réfléchir à la manière d'exécuter une règle.

5.1 Structure des opérateurs

Dans le même esprit que la partie voisinage de ce projet, nous avons décidé d'utiliser une seule classe abstraite **RègleGen** qui est implémenté par tous les opérateurs.

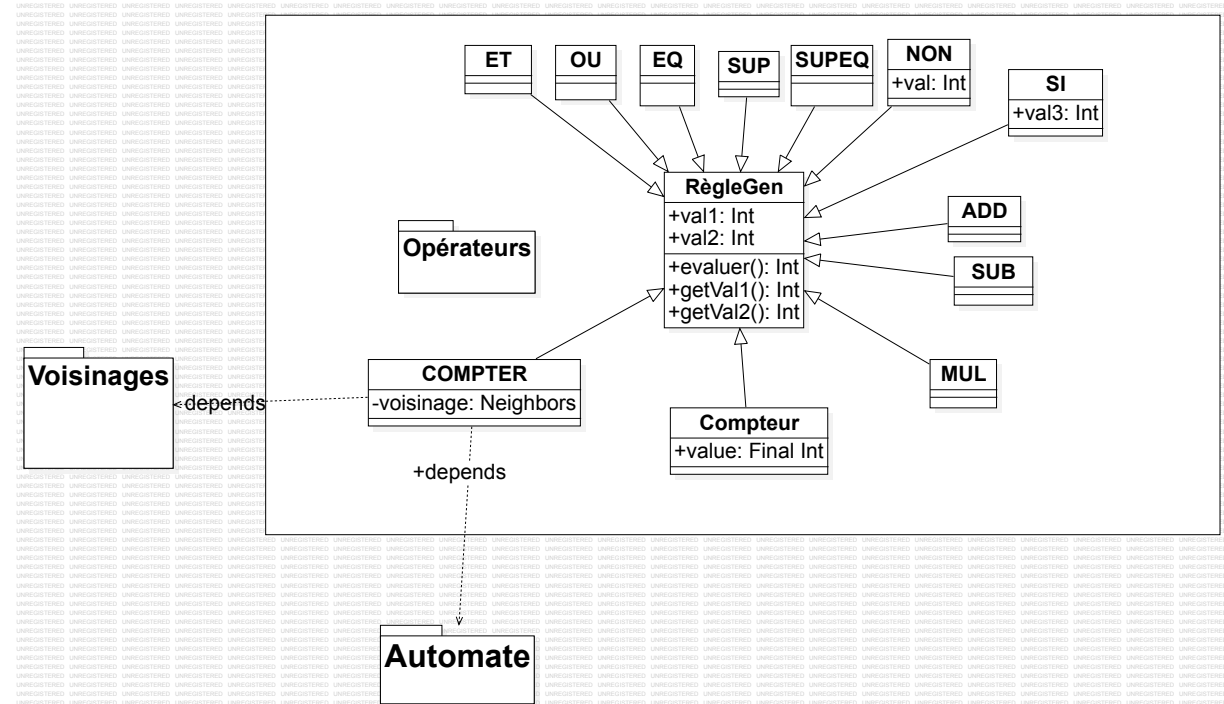


FIGURE 7 – Diagramme de classe du Voisinage

Chacun des opérateurs peuvent être vu comme des nœuds ayant 1,2 ou 3 branches. Dans la construction d'une règle, les branches d'un opérateur seront au fur et mesure rempli de la lecture d'une règle. La classe **RègleGen** contient 2 nœuds, de type int, qui seront initialisés par des appels **super** aux constructeurs mère. La classe **RègleGen** possède une méthode abstraite appelée **evaluer**. Cette opération sera spécifique à l'opérateur et renverra toujours un entier. On a un exemple d'implémentation de cette structure ci-bas.

```

package opérateurs;

public class OU extends RègleGen {
    public OU(int val1, int val2) {
        super(val1, val2);
    }
}
  
```

```
@Override
public int evaluer() {
    return val1 | val2;
}
}
///
public class EQ extends RegleGen {

    public EQ(int val1, int val2) {
        super(val1, val2);
    }

    @Override
    public int evaluer() {
        return val1 == val2 ? 1 : 0;
    }
}
```

5.2 Cas particulier des opérateurs ternaire et unaire

Dans le cas du connecteur **SI**, on a 3 branches au lieu de 2. Il faut donc gérer une branche spécifique à **SI**, en plus de celle de la classe abstraite.

```
public class SI extends RegleGen {
    public int val3;
    public SI(int val1, int val2, int val3) {
        super(val1, val2);
        this.val3=val3;
    }

    @Override
```

```
public int evaluer() {  
    return val1 != 0 ? val2 : val3;  
}  
}
```

Pour les opérateurs unaires comme **NON** ou **COMPTER**. Il faut redéfinir l'unique connecteur. Dans **COMPTER**, le nœud doit toujours être un voisinage **Neighbor**, c'est pourquoi on n'utilise pas `val1` dans cet opérateur. Dans **NON** on appelle un constructeur par défaut dans **RegleGen** et on initialise l'unique nœud `val1` de type entier.

5.3 Principe de lecture d'une règle

Une règle de l'automate est défini dans un fichier de configuration. La règle lue est convertie en tableau de **String**. Pour une règle donnée, il faut être capable de générer une arborescence afin de pouvoir associer correctement chacun des termes d'un connecteur. Une fois que l'arborescence est générée, il faut pouvoir évaluer les résultats d'un nœud. La méthode **évaluer** est commune à l'ensemble des classes qui hérité de **RegleGen** ainsi qu'à la classe **Cellule**. La force du polymorphisme Java fait qu'on peut exécuter la méthode **évaluer** de manière différente en fonction du connecteur qui l'a appelé. On va donc distinguer deux phases :

1. La phase de descente ou on construit l'arborescence
2. La phase de remontée en évaluant les nœuds successive

Les feuilles de notre arbre ne peuvent être que des constantes ou des voisinages (Notée en rouge). Quand on cherche à évaluer un voisinage. La méthode **evaluer** de **COMPTER** va itérer sur chacune des cellules du voisinage et appellera encore **evaluer**. Au niveau d'une cellule, la méthode **evaluer** renvoie 0 ou 1 en fonction de l'état de la cellule (Allumé ou pas). Chacun des opérateurs va remonter dans l'arborescence en recevant le résultat de ses sous opérandes via **evaluer**. C'est ainsi qu'on remonte jusqu'au dernier opérateur qui va renvoyer son résultat.

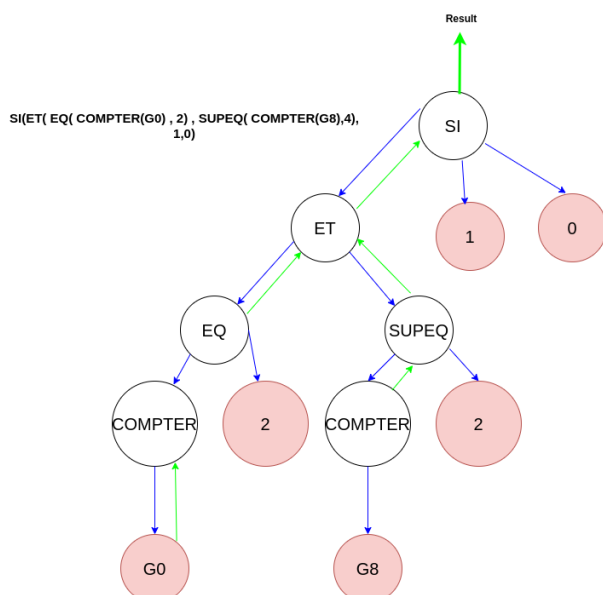


FIGURE 8 – Schéma de l'arborescence d'une règle. En bleu, l'action de descente. En vert, la remontée.

5.4 Implémentation de la solution : Classe TraitRegle

La classe **TraitRegle** va permettre de calculer une règle via ses méthodes.

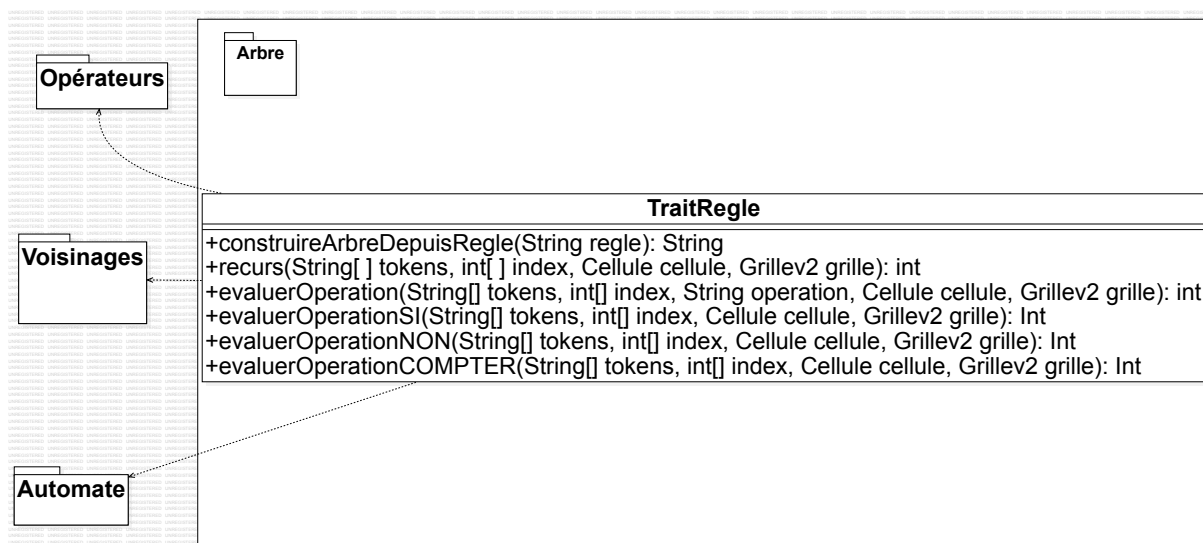


FIGURE 9 – Diagramme de classe de TraitRegle

construireArbreDepuisRegle :

Entrées : **String regle**

Sortie : **String[]**

Fonctionnement : La fonction découpe une chaîne de caractères en une série de tokens, en supprimant les espaces et en conservant les virgules et les parenthèses.

recurs :

Entrées : `String[] tokens`, `int[] index`, `Cellule cellule`, `GrilleV2 grille`

Sortie : `int`

Fonctionnement : La fonction évalue de manière récursive les tokens pour générer des opérations et des valeurs. Les tokens sont récupérés à l'aide d'un curseur interne nommé `index[0]`. On incrémente ce token, ce curseur à chaque fois qu'un token est lu. C'est cette méthode qui génère en profondeur un arbre récursif et qui récupère et renvoie le résultat des opérateurs.

evaluerOperation :

Entrées : `String[] tokens`, `int[] index`, `String operation`, `Cellule cellule`, `GrilleV2 grille`

Sortie : `int`

Fonctionnement : La fonction évalue une opération binaire (ADD, ET, OU, etc.) en analysant les tokens, en récupérant les valeurs des membres gauche et droit, et en appliquant l'opération en fonction du token.

evaluerOperationSI :

Entrées : `String[] tokens`, `int[] index`, `Cellule cellule`, `GrilleV2 grille`

Sortie : `int`

Fonctionnement : La fonction évalue une opération conditionnelle (SI) en analysant les tokens, en récupérant les valeurs condition, alors et sinon, et en appliquant l'opération SI.

evaluerOperationNon :

Entrées : `String[] tokens`, `int[] index`, `Cellule cellule`, `GrilleV2 grille`

Sortie : `int`

Fonctionnement : La fonction évalue une opération NON en analysant les tokens, en récupérant la valeur, et en appliquant l'opération NON.

evaluerOperationCOMPTER :

Entrées : `String[] tokens`, `int[] index`, `Cellule cellule`, `Grille grille`

Sortie : `int`

Fonctionnement : La fonction évalue une opération COMPTER en analysant les tokens, en récupérant le type de voisinage, et en comptant le nombre de cellules dans le voisinage spécifié.

Le fonctionnement global des méthodes **recurs** et **evaluerOperation** peuvent être décrit à l'aide d'un diagramme d'activité que vous pourrez retrouver en **Annexe** de ce rapport.

La fonction **trim** permet de supprimer les espaces superflus avant et après la formule. Les autres méthodes avec un opérateur unaire ou ternaire fonctionnent sur le même principe, il n'attendront pas deux résultats, mais une seule ou trois. Il est à mentionner que la méthode **evaluerOperationCOMPTER** va instancier des voisinages pour les évaluer.

Gestion d'exception

Durant la phase de lecture du tableau **tokens**, on veille à ce que la lecture d'un opérateur soit bien formatée, c'est-à-dire qu'un opérateur puisse être séparé par des parenthèses, ou bien qu'on passe à la lecture d'un autre nœud d'opérateur en lisant une virgule. Pour éviter une mauvaise lecture, on va régulièrement lancer une exception du type **IllegalArgumentException** si jamais l'utilisateur commet une faute de syntaxe.

5.5 Autres

Nous avons la possibilité d'utiliser une bibliothèque nommée **Spring Expression Language (SpEL)**, une fonctionnalité de Spring Framework. Cette bibliothèque nous aurait permis de (...). Cela dit, nous avons estimé qu'il était plus intéressant de coder

nous-mêmes les fonctions nécessaires à la lecture du fichier XML

6 Fichier de configuration

Dans les prérequis de ce sujet, il est mentionné que l'utilisateur doit spécifier sa configuration de d'Automate Cellulaire (Dimension, Voisinages, cellules initialiser, etc.) depuis un fichier de configuration. Grâce aux conseils avisés de nos professeurs, nous avons opté pour le choix d'un fichier de configuration sous format **XML**.

6.1 Rapide introduction à l'XML et avantages d'un tel type de fichiers

L'XML pour (...) est un langage de balisage tout comme l'**HTML** ou le **CSS**. Particulièrement utilisé dans le web pour organiser des pages web, l'**XML** nous permet dans notre projet d'organiser les informations essentielles à l'initialisation pour lancer un Automate Cellulaire :

- La dimension
- La taille
- Les règles
- L'état initial
- Les voisinages utilisés

Pour spécifier ces différentes données, on écrira toute la configuration du fichier **XML** dans une balise nommée **automate**. Ensuite, on utilise de nouvelles balises pour spécifier les différents champs de l'automate.

```
<automate>
<Dimensionalite>2D</Dimensionalite>
<Taille>
  <dimension>30</dimension>
  <dimension>15</dimension>
</Taille>
<Regle>
  SI(EQ(COMPTER(G0), 0), SI(EQ(COMPTER(G8), 3), 1, 0), SI(EQ(COMPTER(G8),
```

```
2), 1, SI(EQ(COMPTER(G8), 3), 1, 0)))  
  
</Regle>  
  
<EtatInitial>RANDOM(15)</EtatInitial>  
  
<Voisinages>  
    <Voisinage>G0</Voisinage>  
    <Voisinage>G1</Voisinage>  
    <Voisinage>G2</Voisinage>  
</Voisinages>  
  
</automate>
```

Cette construction permet d'une part d'ordonner de manière cohérente les informations, car elles sont entourées de balise, mais aussi de faciliter grandement la lecture des informations depuis **Main.java**

6.2 Exploitation au niveau du Main.java

Pour lire un fichier de manière générale et **XML** plus particulièrement, nous avons besoin d'importer un certain nombre de bibliothèques :

- **javax.xml.parsers.DocumentBuilderFactory**, pour obtenir des méthodes qui vont générer un objet de type **DocumentBuilder** qui va analyser le **DOM** du fichier XML
- **javax.xml.parsers.DocumentBuilder**, qui analysera le **DOM** du fichier XML
- **org.w3c.dom.***

On récupère le fichier sous forme d'objet via le constructeur de **File** tout en veillant à bien cibler le fichier **XML**. Ensuite, générer le **DOM** du fichier **XML** en instanciant un objet **DocumentBuilderFactory** puis en utilisant **dbFactory.newDocumentBuilder()** pour obtenir un objet de type **DocumentBuilder**. On va ensuite, lire le **DOM** du fichier **XML** via la méthode **parse** de **dBuilder** et envoyer la référence à un objet de type **Document**. Enfin, on va normaliser l'arborescence du **DOM** en utilisant la méthode **normalize**.

On peut désormais lire les données de configuration du fichier **XML**. En utilisant, l'objet **doc**.

```
// Lire le fichier XML

File inputFile = new File("/home/ing-mustang/Desktop/Sup_Galilee/1er
    anne/Java/Projet_2024/Code/Final/myJavaProject-master4/src/Fichiers/automate.xml");

DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
Document doc = dBuilder.parse(inputFile);
doc.getDocumentElement().normalize();

// Lire les dimensions
NodeList dimensionList = doc.getElementsByTagName("dimension");
ArrayList<Integer> dimensions = new ArrayList<>();
for (int i = 0; i < dimensionList.getLength(); i++) {
    dimensions.add(Integer.parseInt(dimensionList.item(i).getTextContent()));
}
System.out.println(dimensions);

// Lire la règle
String regle =
    doc.getElementsByTagName("Regle").item(0).getTextContent().trim();
System.out.println(regle);

// Lire l'état initial
String etatInitial =
    doc.getElementsByTagName("EtatInitial").item(0).getTextContent().trim();
System.out.println(etatInitial);

// Lire les voisinages
NodeList voisinageList = doc.getElementsByTagName("Voisinage");
```

```
ArrayList<String> voisinages = new ArrayList<>();  
  
for (int i = 0; i < voisinageList.getLength(); i++) {  
    voisinages.add(voisinageList.item(i).getTextContent().trim());  
}
```

7 Interface graphique

L'interface graphique est un aspect incontournable de ce projet, c'est grâce à cette dernière qu'est réalisé le rendu visuel de l'Automate Cellulaire qui est indispensable pour voir l'évolution de l'Automate. Pour ce faire, on va utiliser deux classes. **ColorGrid** et **CellPanel**

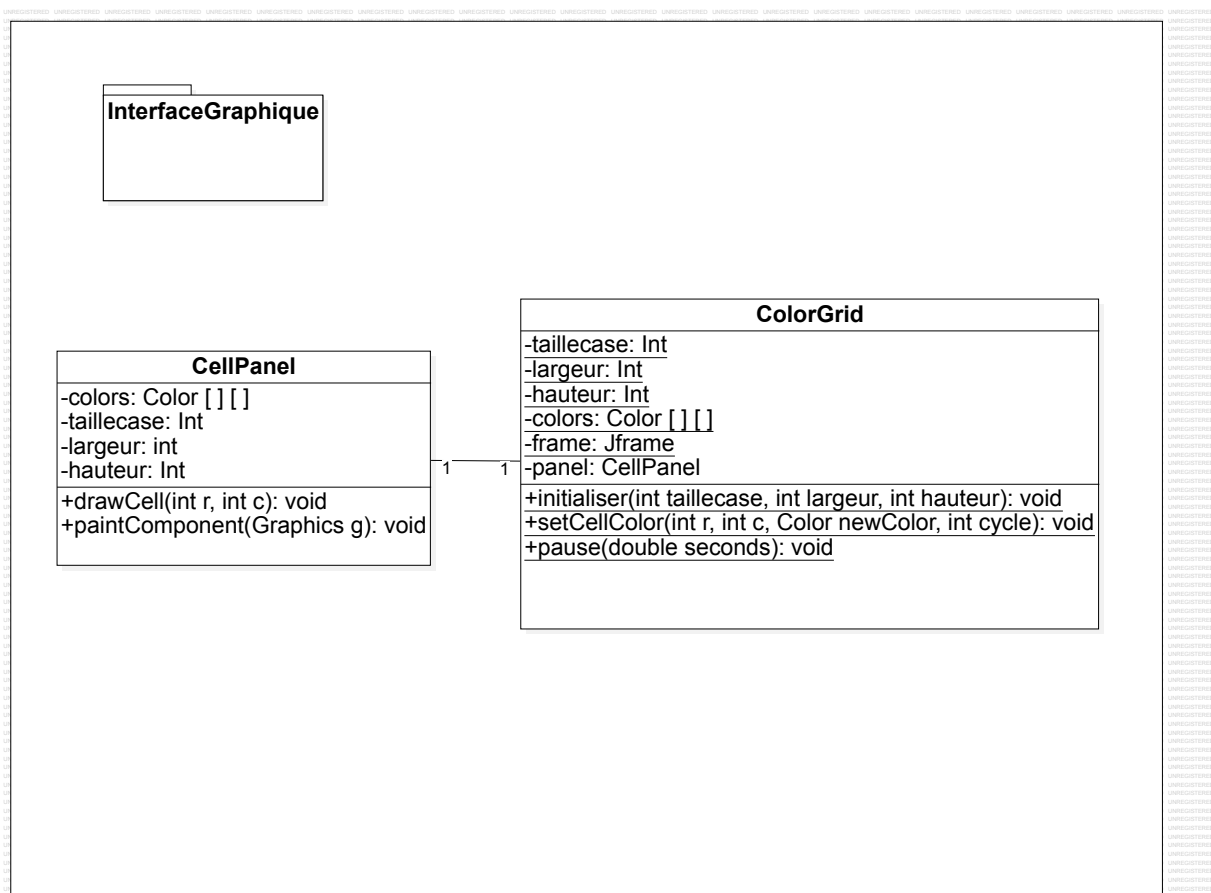


FIGURE 10 – Diagramme de classe du package InterfaceGraphique

Ces deux classes utilisent toutes deux les bibliothèques **javax.swing** et **java.awt**. Ces dernières sont nécessaires pour générer une interface graphique.

La classe **ColorGrid** permet de générer la fenêtre graphique. Elle permet aussi d'afficher dynamiquement le cycle de boucle de l'automate cellulaire et la grille dans son ensemble. L'affichage des cellules est géré par la classe **CellPanel**, elle permet dès les dessins et de les colorer suivant leur état. Dans notre implémentation, une cellule morte est laissée blanche, tandis qu'une cellule vivante est colorée en bleu.

initialiser :

Entrées : `int taillecase, int largeur, int hauteur`

Sortie : `void`

Fonctionnement : Initialise la grille de couleurs avec la taille des cases et les dimensions spécifiées. Crée une fenêtre pour afficher la grille et initialise toutes les cellules avec la couleur blanche.

setCellColor :

Entrées : `int r, int c, Color newColor, int cycle`

Sortie : `void`

Fonctionnement : Met à jour la couleur d'une cellule spécifique dans la grille. Affiche le texte du cycle en haut de la fenêtre. Si les coordonnées de la cellule sont invalides, lance une exception. Si la nouvelle couleur est différente de la couleur précédente, redessine la cellule.

pause :

Entrées : `double seconds`

Sortie : `void`

Fonctionnement : Met en pause l'exécution du programme pendant le nombre de secondes spécifié.

stop :

Entrées : `void`

Sortie : void

Fonctionnement : Ferme la fenêtre graphique et termine l’affichage de la grille.

drawCell :

Entrées : int r, int c

Sortie : void

Fonctionnement : Dessine une cellule à la position spécifiée avec la couleur correspondante. Dessine également la bordure de la cellule.

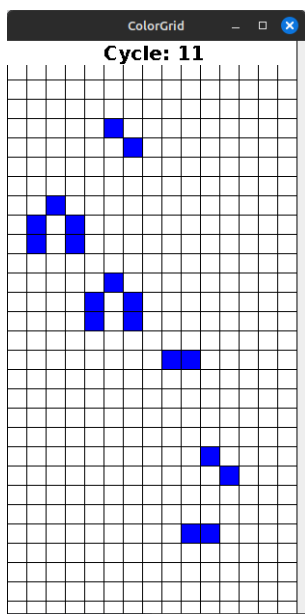
paintComponent :

Entrées : Graphics g

Sortie : void

Fonctionnement : Redéfinit la méthode paintComponent de JPanel pour dessiner la grille de couleurs. Parcourt toutes les cellules et dessine chaque cellule avec la couleur correspondante et une bordure noire.

Au final, notre grille s’affichera de cette manière lors d’une exécution classique.



8 Exécution du Main.java

L'exécution du **Main.java** permet de clore le fonctionnement de notre programme. C'est lui qui va lire le fichier de configuration et lancer la boucle de simulation pour pouvoir lancer l'Automate Cellulaire. L'exécution du programme se fait selon les étapes suivantes :

1. Lecture du fichier de configuration
2. Génération de la grille et des coordonnées
3. Initialisation de l'interface graphique via **taillecase**, ici définit à 20.
4. On initialise de manière aléatoire la grille en la parcourant en profondeur. Si l'utilisateur a choisi d'écrire **RANDOM** dans le champ d'état **initial** du fichier **XML**, on va appliqué de manière aléatoire les états d'une cellule. Sinon, on va mettre à 1, les cellules spécifiées dans le fichier **XML**
5. On va enfin lancer la boucle de simulation/affichage en fonction de la dimension de la grille.

Il existe 3 méthodes pour lancer la simulation, **afficherAutomates1D/2D/3D**, ces méthodes vont lancer une boucle for en fonction du nombre d'itération spécifié dans le fichier main (à savoir 200 cycles). Pour chaque étape du lancement, d'une méthode **afficherAutomate**, on va :

1. Générer une copie du tableau de cellule.
2. Appliquer les règles spécifiées dans le fichier de configuration, pour chaque cellule.
A l'aide d'une boucle for imbriqué. On va ensuite modifier l'état de la cellule via le résultat **result**.

```
for (int i = 0; i < dimensions.get(0); i++) {  
    for (int j = 0; j < dimensions.get(1); j++) {  
        ArrayList<Integer> coord = new  
            ArrayList<>(Arrays.asList(i, j));  
        Cellule c1 = tab.getCellulev2(coord);
```

```
        if (c1 == null || c1.getCoordonnees() == null) {  
            throw new IllegalArgumentException("Cellule c1 ou ses  
                coordonnees sont nulles");  
        }  
  
        String[] tokens =  
            traitRegle.construireArbreDepuisRegle(regle);  
        int result = traitRegle.recurs(tokens, new int[]{0}, c1,  
            tab);  
  
        Cellule newCell = newTab.getCellulev2(coord);  
        newCell.setEtat(result);  
    }  
}
```

3. On va une nouvelle fois itérer sur toute la grille. On va récupérer à chaque tour de boucle, récupérer la cellule issue de la grille copié et de la grille modifié. On va vérifier si c'est les deux cellules ont un états différent. Si telle est le cas, on modifie l'état de la cellule de la grille originale, puis on va modifié l'affichage via **setColor**, la couleur dépendant de l'état de la cellule.
-

```
// Mettre à jour les cellules actuelles à partir de la copie  
for (int i = 0; i < dimensions.get(0); i++) {  
    for (int j = 0; j < dimensions.get(1); j++) {  
        ArrayList<Integer> coord = new  
            ArrayList<>(Arrays.asList(i, j));  
        Cellule c1 = tab.getCellulev2(coord);  
        Cellule newCell = newTab.getCellulev2(coord);  
  
        if (c1.getStatus() != newCell.getStatus()) {  
            c1.setEtat(newCell.getStatus());  
        }  
    }  
}
```



```
        Color newColor = newCell.getStatus() == 1 ?  
            Color.BLUE : Color.WHITE;  
        ColorGrid.setCellColor(i, j, newColor, cycle);  
    }  
}  
}
```

On va enfin utiliser les méthodes **pause** pour **ColorGrid** et **sleep** de **Thread** pour mieux visualiser les changements.

La méthode **AffichageGrille3D**, se comporte de manière analogue à celle de la version 2D (moyennant une boucle for triplement imbriquée). La méthode d'affichage 1D, permet d'afficher successivement les états de la grille sous forme de lignes empilées.

9 Résultats

Notre programme final permet d'afficher l'état d'une grille en fonction du fichier de configuration, bien que son exécution n'est pas adéquat au véritable **Jeu de la Vie**.

9.1 Cas 1D : Triangle de Sierpiński

Pour une grille 1D, si l'on choisit la configuration suivante :

```
<automate>  
  
<Dimensionalite>1D</Dimensionalite>  
  
<Taille>  
    <dimension>30</dimension>  
</Taille>  
  
<Regle> SI(EQ(COMPTER(G4), 1), 1, 0) </Regle> <!--la regle du trinagle de  
    Sierpiski-->  
  
<EtatInitial>(15)</EtatInitial>  
  
<Voisinages>
```

```
<Voisinage>G0</Voisinage>  
<Voisinage>G1</Voisinage>  
<Voisinage>G2</Voisinage>  
</Voisinages>  
</automate>
```

On obtient les figures suivantes :

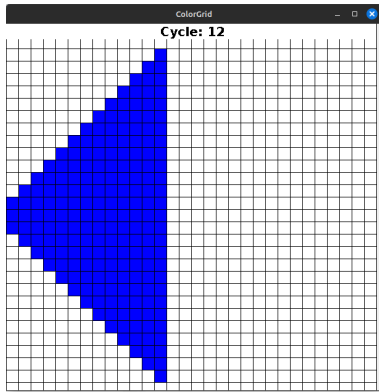


FIGURE 12 – Triangle de Sierpinski à l'état 12

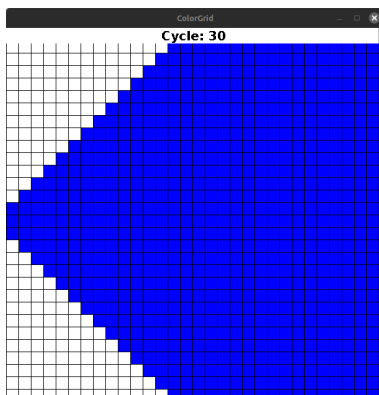


FIGURE 13 – Triangle de Sierpinski à l'état

9.2 Cas 2D

Dans le cas 2D, la configuration suit bien celle qui est spécifiée au début, pour une configuration aléatoire. Ensuite, l'automate évolue, mais s'arrête quand il atteint une configuration stable, qui n'est pas celle observée dans l'algorithme de **John Conway**.

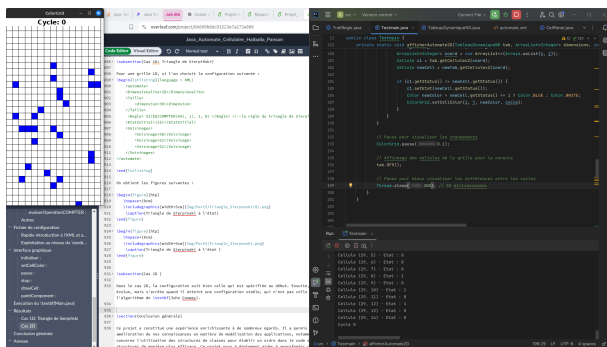


FIGURE 14 – Automate 2D

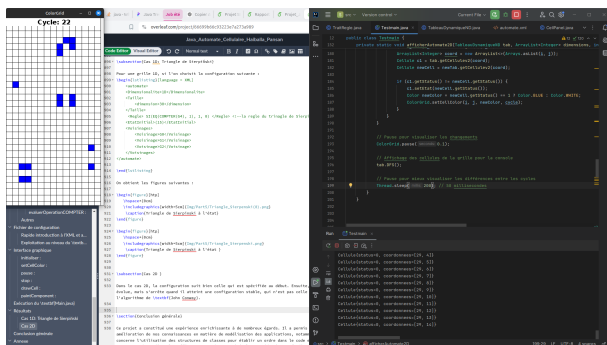


FIGURE 15 – Automate 2D à un état de stabilité

10 Conclusion générale

Au final, de nombreux points de ce projet ont très bien marché, comme la génération de grille, le calcul des voisins, l'interface graphique et le système de règle. Cependant, d'autre aspect de notre code fonctionne moins comme l'exécution de la simulation dans le temps qui produit des résultats assez différents du **Jeu de la vie** originel. Enfin, il aurait pu être judicieux de retravailler la boucle de simulation 2D et 3D, pour gagner, en abstraction, ce qui n'a pu être réalisé par manque de temps.

Ce projet a constitué une expérience enrichissante à de nombreux égards. Il a permis une amélioration de nos connaissances en matière de de programmation orientée objets, notamment en ce qui concerne l'utilisation des structures de classes pour établir un ordre dans le code et le structurer de manière plus efficace. Ce projet nous a également aidé à mieux comprendre comment code de manière plus abstraite et quelles en sont les avantages. Nous avons aussi pu découvrir les fichiers **XML** ce qui peut être très intéressant pour notre formation à venir. Cette expérience a été très formatrice pour nous et nous

sommes reconnaissants d'avoir eu cette opportunité de développement.

11 Annexe

11.1 Diagramme d'activité de TraitRegle

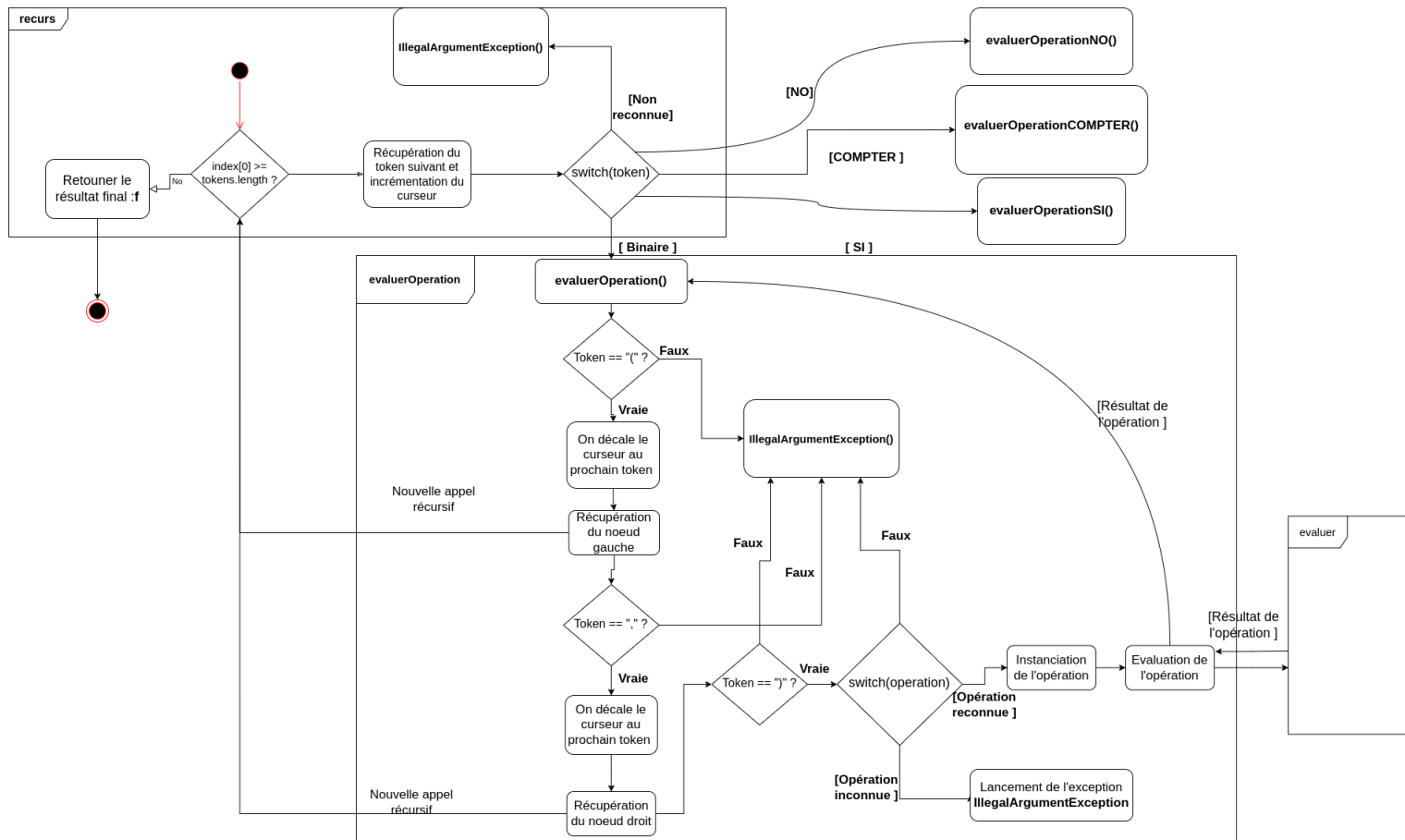


FIGURE 16 – Schéma d'activité de la classe **TraitRegle**